

Developer Productivity Dashboard — Engineering Standards Pack (Task 1)

2026-09-19 · @Someone

1. How to use this pack

Developers edit the mock-based build until every item in the Standards Gate (section 11) passes. This pack is the single source of truth for Task 1, and it is structured so Tasks 2 to 4 extend it without rework.

```
flowchart LR
    A["Read requirement<br/>FR / NFR / UC"] --> B["Edit mock data<br/>or component"]
    B --> C["Run automated<br/>checks"]
    C --> D{"Standards Gate<br/>passes?"}
    D -- No --> B
    D -- Yes --> E["Tag release<br/>task-1-submission"]
```

The loop repeats per requirement: change a mock scenario or a component, run the checks, and fix until the gate is green.

Section	Document	Primary owner	Used for
2	Product Vision and Scope	Product Manager	Agreeing what is and is not built
3	System Analysis	Systems Analyst	Actors, use cases, domain model, rules
4 to 5	SRS (functional, non-functional)	Systems Analyst	Testable requirements
6	UX/UI Handoff Spec	UX/UI Designer	Layout, tokens, states, accessibility
7	Architecture and ADRs	Tech Lead	Structure and TypeScript rules
8	Mock Data and Service Contract	Frontend Engineer	Scenarios developers modify

Section	Document	Primary owner	Used for
9	Test Strategy and Traceability	QA Engineer	Proving each requirement
10	Security and Threat Model	Security Engineer	Frontend risk controls
11	Standards Gate	Whole team	Pass or fail verification
12	Conventions, CI and Roadmap	Tech Lead	Working rules and scaling path

ID conventions. Requirements use `FR-###` and `NFR-###`, use cases `UC-##`, tests `TC-###`, decisions `ADR-###`. Every test names the requirement it proves, and every requirement has at least one test.

2. Product vision and scope

The Developer Productivity Dashboard gives a developer one screen to see what to work on next, how projects are progressing, and what is overdue. Task 1 delivers the complete frontend on mock data; Tasks 2 to 4 replace the mocks with a real API, database, and an AI feature.

Vision. A fast, accessible, responsive workspace that a small product team could ship as-is.

Users

Persona	Goal	Key screens
Individual developer	See today's tasks and progress at a glance	Dashboard, Tasks
Team lead	Track project health and spot overdue work	Dashboard, Projects
Evaluator / reviewer	Confirm quality, states, and responsiveness quickly	All, with scenario switcher

Success metrics

Metric	Target
Lighthouse (performance, accessibility, best practices)	90 or higher each, mobile profile
axe automated accessibility violations	0 on every route
Guide requirements covered by a passing test	100%
TypeScript strict errors, <code>any</code> usage, <code>.js</code> files in repo	0

In scope (Task 1)

- Dashboard, projects, tasks, and profile screens on mock data
- Search, filter, sort, and progress indicators
- Loading, empty, and error states on every dynamic view
- Light and dark themes, responsive at mobile, tablet, and desktop

Out of scope (Task 1)

- Real authentication, persistence, and server APIs (Tasks 2 and 3)
- AI features (Task 4)
- Multi-user collaboration and real-time updates

Deliverables from the guide: GitHub repository, demo video, LinkedIn post. The evaluation weights responsiveness, component structure, state handling, and visual consistency.

3. System analysis

The system has one human actor and one external boundary in Task 1: the developer, and a data service that is mocked now and becomes the FastAPI backend in Task 2.

Actors

Actor	Type	Responsibility
Developer (User)	Human	Views dashboard, manages tasks, edits own profile

Actor	Type	Responsibility
Team Lead	Human, same UI with wider view	Views all projects and team progress
Data Service	System	Supplies projects, tasks, users (mock adapter now, REST API later)

Use cases

ID	Use case	Actor	Main flow
UC-01	View dashboard summary	User	Open app, see KPIs, deadlines, recent activity
UC-02	Browse projects	User	Open Projects, see cards with progress
UC-03	Browse tasks	User	Open Tasks, see task cards grouped or listed
UC-04	Search tasks and projects	User	Type in search, list narrows, empty state if none
UC-05	Filter and sort	User	Choose status, priority, project, sort order
UC-06	Inspect a project	User	Open project, see its tasks and progress ring
UC-07	Update a task's status	User	Change status, progress recalculates
UC-08	View and edit profile	User	Open profile, edit name and preferences, validate, save
UC-09	Recover from a failed load	User	See error state, press Retry
UC-10	Switch theme	User	Toggle light, dark, or system

Domain model

```
erDiagram
    USER ||--o{ PROJECT : owns
    PROJECT ||--o{ TASK : contains
    USER ||--o{ TASK : assigned
    USER ||--o{ ACTIVITY : performs
    PROJECT ||--o{ ACTIVITY : logs
    USER {
        string id
        string name
        string email
        string role
    }
    PROJECT {
        string id
        string name
        string status
        date dueDate
    }
    TASK {
        string id
        string title
        string status
        string priority
        date dueDate
    }
```

The entity fields above are the minimum; full field lists and types are in section 8.

Business rules

ID	Rule
BR-01	Task status is one of <code>todo</code> , <code>in_progress</code> , <code>in_review</code> , <code>done</code> .
BR-02	Task priority is one of <code>low</code> , <code>medium</code> , <code>high</code> , <code>urgent</code> .
BR-03	Project progress = done tasks divided by total tasks, rounded to a whole percent; a project with no tasks shows 0% and an empty state.

ID	Rule
BR-04	A task is overdue when its due date is before today and its status is not done .
BR-05	Search matches title and description, case-insensitive, and ignores leading and trailing spaces.
BR-06	Filters combine with AND; multiple values inside one filter combine with OR.

Task status workflow

```
stateDiagram-v2
    [*] --> todo
    todo --> in_progress
    in_progress --> in_review
    in_review --> in_progress
    in_review --> done
    done --> in_progress
```

4. SRS: functional requirements

There are 24 functional requirements; each item in the guide’s Task 1 list is covered by at least one, and the rest raise the build to production standard. The guide’s reusable-component item is governed by section 7. Priority uses MoSCoW: M = must, S = should, C = could.

ID	Requirement	Pri	Acceptance criteria	Use case
FR-01	Dashboard is the default landing route /	M	Opening / shows KPIs, deadlines, and activity without extra navigation	UC-01
FR-02	Dashboard shows four KPIs: active projects, open tasks, overdue tasks, completion rate	M	Values match mock data and BR-03, BR-04	UC-01

ID	Requirement	Pri	Acceptance criteria	Use case
FR-03	Dashboard lists upcoming deadlines (next 7 days) sorted soonest first	S	List excludes done tasks; empty state when none	UC-01
FR-04	Dashboard shows a recent activity feed (latest 10)	S	Newest first, relative times, empty state	UC-01
FR-05	Navigation bar with links to Dashboard, Projects, Tasks, Profile	M	Current page marked with <code>aria-current="page"</code> ; reachable by keyboard	UC-01 to 08
FR-06	Navigation is a sidebar at 1024px and wider and a drawer below 1024px	M	Drawer opens with a button, traps focus, closes on Escape and on link selection	UC-01
FR-07	Skip-to-content link is the first focusable element	M	Visible on focus and moves focus to <code>main</code>	UC-01
FR-08	Profile menu in header with avatar, name, theme toggle	M	Opens by click and keyboard; closes on Escape	UC-10
FR-09	Profile page shows avatar, name, email, role, personal stats	M	Stats derive from that user's tasks	UC-08
FR-10	Profile form edits name and preferences with inline validation	S	Invalid input shows a message linked by <code>aria-describedby</code> ; save disabled while invalid or saving	UC-08
FR-11	Project cards show name, status, due date, progress, task counts	M	All fields render for every mock project; long names truncate with a title tooltip	UC-02
FR-12	Task cards show title, status, priority, due date, assignee, project	M	Overdue tasks are visibly marked and announced to screen readers	UC-03
FR-13	Progress indicator component (bar and ring) with percentage	M	Uses <code>role="progressbar"</code> with <code>aria-valuenow</code> , <code>aria-valuemin</code> , <code>aria-valuemax</code>	UC-02, 06

ID	Requirement	Pri	Acceptance criteria	Use case
FR-14	Project detail view lists that project's tasks and progress	S	Route <code>/projects/[id]</code> ; unknown id shows a not-found state	UC-06
FR-15	Search box filters projects and tasks by text (BR-05)	M	Debounced 250 ms; result count announced via a live region	UC-04
FR-16	Filters: status, priority, project; sort: due date, priority, title	M	Combine per BR-06; a clear-all control resets them	UC-05
FR-17	Search, filter, and sort state is stored in the URL query string	S	Reloading or sharing the URL restores the same view	UC-04, 05
FR-18	No-results state distinct from no-data state	M	No-results offers Clear filters; no-data offers a create-first action	UC-04
FR-19	Change a task's status from its card	S	Optimistic update, rolls back and shows an error toast on failure	UC-07
FR-20	Loading state: skeletons matching final layout on every dynamic view	M	No layout shift when data arrives (CLS under 0.1)	UC-01 to 06
FR-21	Empty state on every list and feed, with icon, message, and action	M	Each empty state is reachable through a mock scenario	UC-01 to 06
FR-22	Error state on every dynamic view with a Retry action	M	Retry refetches; error text is plain language with no stack traces	UC-09
FR-23	Fully responsive at 360, 768, and 1280 px and wider, with no horizontal page scroll	M	Verified by automated viewport tests	All
FR-24	Light, dark, and system themes, persisted per browser	S	No flash of wrong theme on load	UC-10

Every Must requirement blocks submission. Should requirements are expected and tracked in the gate but do not block.

5. SRS: non-functional requirements

Every non-functional requirement has a number and a tool that measures it, so a developer knows when it is met.

ID	Category	Requirement	Target	Verified by
NFR-01	Performance	Largest Contentful Paint on mobile profile	2.5 s or less	Lighthouse CI
NFR-02	Performance	Cumulative Layout Shift	Under 0.1	Lighthouse CI
NFR-03	Performance	Interaction to Next Paint	200 ms or less	Lighthouse CI, Playwright trace
NFR-04	Performance	JavaScript shipped on first load per route (gzip)	170 KB or less	Bundle analyzer in CI
NFR-05	Accessibility	WCAG 2.2 level AA on every route	0 axe violations	axe in Playwright
NFR-06	Accessibility	Full keyboard operation, visible focus, logical tab order	All flows operable without a mouse	Manual checklist plus Playwright
NFR-07	Accessibility	Text contrast 4.5:1, UI component contrast 3:1, in both themes	No failures	Contrast test on tokens
NFR-08	Accessibility	Respects <code>prefers-reduced-motion</code>	Animations off or minimal	Component test
NFR-09	Responsiveness	Supported widths	360 px to 2560 px	Playwright viewports
NFR-10	Responsiveness	Touch targets	44 by 44 px minimum on touch layouts	Visual test

ID	Category	Requirement	Target	Verified by
NFR-11	Type safety	TypeScript strict mode, zero any , zero .js or .jsx files	0 violations	tsc --noEmit , ESLint, file check in CI
NFR-12	Maintainability	Component size and layering	No component over 200 lines; UI layer imports no feature code	ESLint boundary rule
NFR-13	Maintainability	Code style consistent	Prettier and ESLint clean	CI lint step
NFR-14	Testability	Unit and component coverage on src/	80% lines or higher	Vitest coverage
NFR-15	Security	No secrets in client code or repo	0 findings	Secret scan in CI
NFR-16	Security	Content Security Policy and security headers set	A rating or higher	Headers test
NFR-17	Security	Dependencies free of known high or critical vulnerabilities	0 findings	npm audit in CI
NFR-18	Compatibility	Latest two versions of Chrome, Edge, Firefox, Safari	All flows pass	Playwright projects
NFR-19	Resilience	Any single failed request leaves the rest of the page usable	Error isolated to its region	Error-boundary test
NFR-20	Observability	Client errors captured through one reporting function	Single interface, swappable	Unit test

ID	Category	Requirement	Target	Verified by
NFR-21	Internationalisation	User-facing strings kept out of components	Strings in one dictionary	Lint rule and review
NFR-22	Deployability	Reproducible build and preview deployment	Fresh clone builds with 3 commands	CI
NFR-23	Documentation	README covers setup, scripts, architecture, and how to add a screen	Follows README template	Review
NFR-24	Portability	Backend swap needs no component change	Only the data adapter changes	Contract tests

6. UX/UI handoff spec

The interface uses four routes inside one app shell, a token-based design system, and a fixed set of states for every dynamic region. Designers own the tokens and layouts; developers implement them without hard-coded values.

Information architecture

```

flowchart TD
    Shell["App shell<br/>header + navigation"] --> D["Dashboard /"]
    Shell --> P["Projects /projects"]
    Shell --> T["Tasks /tasks"]
    Shell --> U["Profile /profile"]
    P --> PD["Project detail<br/>/projects/[id]"]
  
```

Responsive layout

Breakpoint	Width	Navigation	Dashboard grid	Card list
Mobile	360 to 639 px	Drawer, bottom-safe actions	1 column, KPIs 2 by 2	1 column

Breakpoint	Width	Navigation	Dashboard grid	Card list
Tablet	640 to 1023 px	Drawer	2 columns	2 columns
Desktop	1024 to 1439 px	Fixed sidebar 256 px	12-column grid	3 columns
Wide	1440 px and up	Fixed sidebar 256 px, content max width 1280 px	12-column grid	4 columns

Design tokens (defined once as CSS variables and mapped in the Tailwind config; no raw hex or pixel values in components)

Token group	Definition
Colour	Brand gradient cyan to blue to violet (from the Innovation Hacks guide); neutral scale 50 to 950; semantic success, warning, danger, info; light and dark sets
Typography	One sans family with system fallback; scale 12, 14, 16, 18, 20, 24, 30, 36 px; weights 400, 500, 600, 700; line height 1.5 body, 1.2 headings
Spacing	4 px base unit; steps 4, 8, 12, 16, 24, 32, 48, 64
Radius and elevation	Radius 6, 10, 16 px; three shadow levels
Motion	150 ms and 250 ms durations; ease-out; disabled under reduced motion

Component inventory (built once, reused everywhere)

Layer	Components
Primitives	Button, IconButton, Input, Select, Checkbox, Badge, Avatar, Tooltip, Skeleton, Spinner, Toast
Composites	Card, ProgressBar, ProgressRing, SearchField, FilterBar, SortMenu, Drawer, DropdownMenu, EmptyState, ErrorState

Layer	Components
Features	ProjectCard, TaskCard, KpiTile, DeadlineList, ActivityFeed, ProfileForm, ThemeToggle
Layout	AppShell, Header, Sidebar, PageHeader, Grid

State matrix. Every dynamic region implements all five states; none may be skipped.

State	Behaviour	Accessibility
Loading	Skeletons the same size as final content	Region marked <code>aria-busy="true"</code>
Success	Data displayed	Headings and landmarks in order
Empty (no data)	Icon, one-line explanation, primary action	Message in reading order
No results	Message, active filters shown, Clear filters action	Count announced in live region
Error	Plain-language message, Retry button, region-only impact	<code>role="alert"</code> , focus moves to Retry on first failure

Accessibility rules for design and code

- One `h1` per page, headings never skip levels, landmarks `header` , `nav` , `main` .
- Colour is never the only signal: status and priority also carry text or an icon.
- Focus ring 2 px, offset 2 px, contrast 3:1 or higher against adjacent colours.
- Forms use visible labels, not placeholders alone; errors name the field and the fix.

Design handoff checklist: tokens exported, all four breakpoints mocked, all five states mocked per region, dark theme mocked, focus and hover states shown, empty-state copy written.

7. Architecture, TypeScript rules and decisions

The frontend is a layered Next.js application in strict TypeScript, where components depend on a service interface and never on mock data directly. That single seam lets Task 2 swap in the FastAPI backend by changing one adapter.

Stack

Concern	Choice
Framework	Next.js (App Router) with React, TypeScript strict
Styling	Tailwind CSS driven by design tokens
Server state	TanStack Query (caching, loading, error, retry)
Runtime validation	Zod schemas at the service boundary
Accessible primitives	Radix UI headless components
Testing	Vitest, Testing Library, Playwright, axe-core
Quality	ESLint (typescript-eslint strict), Prettier, <code>tsc --noEmit</code>

Layers and allowed dependencies

```
flowchart TD
  A["app/ routes"] --> F["features/"]
  F --> U["ui/ primitives"]
  F --> S["services/ interfaces"]
  S --> M["adapters/mock"]
  S -. -> H["adapters/http (Task 2)"]
  U --> T["tokens / lib"]
```

Dependencies point downward only. `ui/` imports nothing from `features/` or `services/`, and an ESLint boundary rule fails the build if that is broken.

Folder structure

```
frontend/  
  src/  
    app/                routes, layouts, loading.tsx, error.tsx, not-  
found.tsx  
    features/  
      dashboard/  projects/  tasks/  profile/  
    ui/            primitives and composites (no business logic)  
    services/      ProjectService, TaskService, UserService interfaces  
    adapters/  
      mock/        data, scenarios, latency and failure injection  
      http/        added in Task 2  
    schemas/       Zod schemas and inferred types  
    lib/           formatting, dates, a11y helpers, error reporting  
    i18n/          string dictionary  
    styles/       tokens.css, globals.css  
    tests/         unit, component, e2e, a11y  
    docs/          this pack, ADRs
```

TypeScript rules (enforced, not advisory)

Rule	Setting
Strictness	<code>strict: true</code> , <code>noUncheckedIndexedAccess</code> , <code>exactOptionalPropertyTypes</code> , <code>noImplicitOverride</code>
No JavaScript source	<code>allowJs: false</code> ; CI fails if any <code>.js</code> or <code>.jsx</code> file exists outside the allowlist
No <code>any</code>	<code>@typescript-eslint/no-explicit-any</code> as error; use <code>unknown</code> and <code>narrow</code>
Types come from schemas	Zod schema first, <code>z.infer</code> for the type; no hand-written duplicates
Exhaustive unions	Status and priority are string unions with <code>never</code> exhaustiveness checks
Config in TypeScript	<code>next.config.ts</code> , <code>vitest.config.ts</code> , <code>playwright.config.ts</code> , <code>eslint.config.ts</code>
Allowlist	Only files the toolchain accepts solely as JavaScript, currently <code>postcss.config.mjs</code> ; declarative, no logic

Architecture decision records

ADR	Decision	Reason
ADR-001	Next.js App Router, with Vite plus React as the documented alternative	Guide-approved; layouts, loading and error files map to required states
ADR-002	TypeScript strict everywhere	Guide-approved; removes a class of runtime defects
ADR-003	Service interface with mock adapter	Backend does not exist yet; keeps components unchanged in Task 2
ADR-004	TanStack Query for server state	Gives loading, error, retry, and cache without custom code
ADR-005	Filter state in the URL	Shareable views and restore on reload

ADR	Decision	Reason
ADR-006	Radix primitives for menus, dialogs, drawers	Correct keyboard and ARIA behaviour by default
ADR-007	Design tokens as CSS variables mapped in Tailwind	One place to change brand and theme
ADR-008	Types generated from the FastAPI OpenAPI schema from Task 2 onward	One contract shared by Python backend and TypeScript client

8. Mock data and service contract

Developers change scenarios and fixtures, not components, to prove every state works. Mocks live in `adapters/mock/` and implement the same interfaces the real API will implement in Task 2.

Entities

Entity	Fields (type)
User	<code>id</code> (string), <code>name</code> (string), <code>email</code> (string), <code>role</code> (<code>developer</code> or <code>lead</code>), <code>avatarUrl</code> (string, optional), <code>preferences.theme</code> (<code>light</code> , <code>dark</code> , <code>system</code>)
Project	<code>id</code> , <code>name</code> (1 to 80 chars), <code>description</code> , <code>status</code> (<code>planned</code> , <code>active</code> , <code>on_hold</code> , <code>completed</code>), <code>dueDate</code> (ISO date), <code>ownerId</code>
Task	<code>id</code> , <code>projectId</code> , <code>title</code> (1 to 120 chars), <code>description</code> , <code>status</code> (BR-01), <code>priority</code> (BR-02), <code>dueDate</code> , <code>assigneeId</code>
Activity	<code>id</code> , <code>actorId</code> , <code>projectId</code> , <code>taskId</code> (optional), <code>type</code> (<code>created</code> , <code>status_changed</code> , <code>completed</code>), <code>at</code> (ISO datetime)

Contract shape (illustrative; the schemas file is the source of truth)

```
export const TaskStatus = z.enum(["todo", "in_progress", "in_review",
"done"]);
export type TaskStatus = z.infer<typeof TaskStatus>;

export interface TaskService {
  list(query: TaskQuery, signal?: AbortSignal): Promise<Page<Task>>;
  updateStatus(id: string, status: TaskStatus): Promise<Task>;
}

export type ApiError = { error: { code: string; message: string } };
```

The error envelope matches the one the Task 2 FastAPI backend returns, so the http adapter needs no translation layer. Every adapter response is parsed with its Zod schema, and a parse failure is treated as an error state.

Scenarios (selected with `?scenario=<name>` in development, or the dev-only scenario switcher)

Scenario	What it proves	Requirements
default	Realistic mixed data	FR-01 to FR-14
loading	Requests take 3 s, skeletons hold layout	FR-20
empty	Zero projects, tasks, activity	FR-18, FR-21
error	Every list request fails with 500	FR-22, NFR-19
partial-error	Only the tasks request fails, rest of page works	NFR-19
flaky	First request fails, retry succeeds	FR-22
update-fails	Status change returns error and rolls back	FR-19
large	500 tasks, 40 projects, scrolling and search stay fast	NFR-01, NFR-03
edge-text	80-character names, long words, emoji, right-to-left text	FR-11, FR-12

Minimum fixture set for default : 4 users, 8 projects (one with no tasks, one completed, one on hold), 60 tasks spread across all statuses and priorities, 40 activity items. Dates are computed relative to today, so overdue, due-today, and upcoming items stay present whenever the app runs.

Mock rules

- Latency defaults to 300 to 900 ms random, configurable, and 0 in unit tests.
- Mock state is in memory and resets on reload; no browser storage holds task data.
- Components import services from a single provider and never import from `adapters/`.
- Adding a scenario means one file plus one row in the table above and a test that references it.

9. Test strategy and traceability

Automated tests carry most of the proof, with manual checks reserved for what tools cannot judge. Each guide requirement traces to functional requirements and to named tests, so a failing test points straight to what is broken.

Test levels

Level	Tool	Scope	Runs
Static	<code>tsc --noEmit</code> , ESLint, Prettier, secret scan	Types, style, boundaries, no <code>.js</code> files	Every commit
Unit	Vitest	Pure logic: progress, overdue, filters, sort, formatters	Every commit
Component	Vitest and Testing Library	Each UI component across its states and props	Every commit
End to end	Playwright	Full flows per scenario, at 3 viewports, 4 browsers	Every pull request
Accessibility	axe-core in Playwright	Every route, both themes, every scenario	Every pull request
Performance	Lighthouse CI	Mobile profile on <code>/</code> , <code>/projects</code> , <code>/tasks</code> , <code>/profile</code>	Every pull request
Manual	Checklist	Keyboard walkthrough, screen reader smoke test, visual review	Before tagging

Traceability matrix

Guide requirement	Requirements	Tests	Level
Dashboard as landing view	FR-01 to FR-04	TC-001 landing route, TC-002 KPI calculations, TC-003 deadlines and feed	E2E, unit
Accessible navigation	FR-05 to FR-08, NFR-06	TC-010 aria-current, TC-011 keyboard and drawer focus trap, TC-012 skip link	Component, E2E
User and profile section	FR-09, FR-10	TC-020 profile stats, TC-021 form validation and messages	Component, E2E
Project and task cards, consistent visuals	FR-11, FR-12, NFR-07	TC-030 card fields and truncation, TC-031 overdue marking, TC-032 visual snapshots	Component, visual
Progress indicators	FR-13, BR-03	TC-040 progress math, TC-041 progressbar ARIA	Unit, component
Search and filter	FR-15 to FR-18, BR-05, BR-06	TC-050 search matching, TC-051 filter combination, TC-052 sort, TC-053 URL restore, TC-054 no-results vs no-data	Unit, E2E
Fully responsive	FR-23, NFR-09, NFR-10	TC-060 no horizontal scroll at 360, 768, 1280, 2560 px, TC-061 touch target size	E2E
Loading and empty states (plus error)	FR-20 to FR-22, NFR-19	TC-070 loading, TC-071 empty, TC-072 error and retry, TC-073 partial error, one per scenario	E2E
Clean reusable component architecture	NFR-11 to NFR-13	TC-080 boundary lint, TC-081 strict types, TC-082 no JavaScript files, TC-083 component size	Static

Guide requirement	Requirements	Tests	Level
Quality bar	NFR-01 to NFR-05, NFR-15 to NFR-17	TC-090 Lighthouse budgets, TC-091 axe zero violations, TC-092 secret scan, TC-093 dependency audit	CI

Rules for tests

- A test's title starts with its `TC-###` and names the requirement it proves.
- Each scenario in section 8 has at least one end-to-end test that fails if the state is missing.
- Tests use role and label queries, not CSS selectors, which also proves accessible names exist.
- A bug fix adds a test that failed before the fix.

10. Frontend security and threat model

Task 1 has no real users or server data, so the goal is to build habits and controls that stay correct when the backend arrives. The main risks are script injection, unsafe URL state, leaked secrets, and vulnerable dependencies.

Assets and trust boundaries. Assets: user profile data, task and project data, and, from Task 2, session credentials. Untrusted inputs: form fields, the URL query string, and every API response until it passes schema validation.

Threats and controls (STRIDE categories in brackets)

ID	Threat	Control	Verified by
TH-01	Cross-site scripting through rendered text [Tampering]	React escaping by default; <code>dangerouslySetInnerHTML</code> banned by lint; strict CSP	Lint rule, headers test
TH-02	Malicious query parameters alter behaviour [Tampering]	Parse all URL state with Zod; unknown values fall back to defaults	Unit test on URL parser

ID	Threat	Control	Verified by
TH-03	Malformed or hostile API response breaks UI [Denial of service]	Zod validation at the adapter; parse failure becomes the error state	Contract tests
TH-04	Secrets committed or shipped to the browser [Information disclosure]	No secrets in the repo; only non-sensitive <code>NEXT_PUBLIC_</code> variables; secret scan in CI	TC-092
TH-05	Vulnerable or malicious dependency [Tampering]	Lockfile committed, exact versions, <code>npm audit</code> gate, automated update pull requests, no unused packages	TC-093
TH-06	Clickjacking [Spoofing]	<code>frame-ancestors 'none'</code> in CSP and <code>X-Frame-Options: DENY</code>	Headers test
TH-07	Error messages leak internals [Information disclosure]	Plain-language messages only; details go to the reporting function, never the screen	Component test
TH-08	Tokens stolen from browser storage [Information disclosure]	Task 2 onward: session in an <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite</code> cookie; nothing sensitive in <code>localStorage</code>	Review at Task 2 gate
TH-09	Third-party scripts run with page privileges [Elevation of privilege]	No third-party scripts; fonts and assets self-hosted	CSP report
TH-10	Prompt injection through AI feature [Tampering]	Task 4: treat model output as untrusted text, never as HTML; server-side guardrails	Task 4 gate

Required response headers

Header	Value
<code>Content-Security-Policy</code>	<code>default-src 'self'; script-src 'self' 'nonce-...'; style-src 'self'; img-src 'self' data;; frame-ancestors 'none'; base-uri 'self'; form-action 'self'</code>
<code>X-Content-Type-Options</code>	<code>nosniff</code>

Header	Value
Referrer-Policy	strict-origin-when-cross-origin
Permissions-Policy	Camera, microphone, geolocation disabled
Strict-Transport-Security	max-age=63072000; includeSubDomains on the deployed site

The controls map to OWASP Top 10 and ASVS; the AI feature will additionally be reviewed against the OWASP Top 10 for LLM applications in Task 4.

11. Standards Gate

The build is submission-ready only when every Must item below passes; anything failing goes back into the edit loop from section 1. Part A is a single command, and parts B to F are checked by a reviewer who did not write the code.

A. Automated gate

```
npm run gate    # runs all of the following in order and stops at the first failure
```

```
npm run typecheck    # tsc --noEmit, strict
npm run lint          # ESLint, Prettier, boundary and no-any rules
npm run check:no-js   # fails on any .js/.jsx outside the allowlist
npm run test          # Vitest unit and component, coverage 80% or higher
npm run test:e2e      # Playwright, 3 viewports, all scenarios
npm run test:a11y     # axe, zero violations, both themes
npm run lighthouse    # budgets from NFR-01 to NFR-04, scores 90 or higher
npm run audit         # no high or critical vulnerabilities, secret scan clean
npm run build         # production build succeeds with no warnings
```

B. Functional review (Must items from section 4)

- ☐ / opens the dashboard with four correct KPIs, deadlines, and activity (FR-01, FR-02)
- ☐ Navigation reaches all four routes by mouse and by keyboard; current page is marked (FR-05)
- ☐ Drawer opens, traps focus, closes on Escape at widths under 1024 px (FR-06)
- ☐ Skip link is first in tab order and works (FR-07)
- ☐ Profile page and menu show correct data; theme toggle works (FR-08, FR-09)

- ☐ Project and task cards show every field; long text truncates cleanly (FR-11, FR-12)
- ☐ Progress bars and rings match the calculated percentage (FR-13)
- ☐ Search, filters, and sort combine correctly and can be cleared (FR-15, FR-16)
- ☐ No-results and no-data states are different and each has the right action (FR-18)

C. States review

- ☐ Each of the nine scenarios in section 8 is reachable and renders the expected state
- ☐ No layout shift when skeletons are replaced by data
- ☐ Retry works from every error state, and a failed region never blanks the page (FR-22, NFR-19)

D. Responsive review

- ☐ 360, 768, 1280, and 2560 px show no horizontal page scroll, clipped text, or overlap (FR-23)
- ☐ Touch targets are 44 px or larger on touch layouts (NFR-10)
- ☐ Layout reflows correctly at 200% browser zoom

E. Accessibility review

- ☐ Every flow completes with the keyboard only; focus is always visible (NFR-06)
- ☐ Screen reader smoke test on dashboard and task list: landmarks, headings, and status announcements make sense
- ☐ Reduced-motion setting removes animation (NFR-08)
- ☐ Colour is never the only signal for status or priority

F. Design consistency review

- ☐ No raw colour, spacing, or font values outside the token files
- ☐ All cards, buttons, inputs, and badges come from `ui/` with no local restyling
- ☐ Light and dark themes both meet contrast requirements (NFR-07)

G. Delivery

- ☐ README follows the template, and a fresh clone runs in 3 commands (NFR-22, NFR-23)
- ☐ Any relaxation of the CSP or other rule is recorded as an ADR
- ☐ Commits follow `type(scope): description` with scope `frontend`
- ☐ Git tag `task-1-submission` created on the passing commit
- ☐ Demo video recorded, showing each scenario and the responsive layouts

- ☐ LinkedIn post published

Failure handling. A failed item gets a defect note naming the requirement ID, the fix, and the test that now covers it; the gate is then rerun in full.

12. Conventions, CI and scaling roadmap

The same gate that developers run locally runs in CI on every pull request, so the standard cannot be skipped. The roadmap then shows what each later task changes, so the team scales this build rather than restarting it.

Repository and Git conventions

Topic	Rule
Layout	Monorepo folders <code>frontend/</code> , <code>backend/</code> , <code>database/</code> , <code>docs/</code> ; Task 1 lives in <code>frontend/</code>
Branching	<code>task/1-frontend</code> for the work; short-lived feature branches merged by pull request
Commits	<code>type(scope): description</code> ; scopes <code>frontend</code> , <code>api</code> , <code>db</code> , <code>ai</code> , <code>auth</code> , <code>deploy</code> , <code>env</code> , <code>docs</code> ; committed file by file in logical groups, not in one bulk commit
Releases	Tag <code>task-1-submission</code> on the commit that passes the gate; later tags follow the same pattern
Environment	<code>.env.example</code> committed; real <code>.env</code> files never committed

CI pipeline

flowchart LR

```

A["Push or<br/>pull request"] --> B["Static checks<br/>types, lint, no-JS"]
B --> C["Unit and<br/>component tests"]
C --> D["Production<br/>build"]
D --> E["E2E and<br/>accessibility"]
E --> F["Lighthouse<br/>and audit"]
F --> G["Preview<br/>deployment"]

```

Each stage blocks the next, and a pull request cannot merge until all stages pass. Preview deployments run on the host chosen for Task 1 (Vercel is the default for Next.js).

Pull request definition of done: requirement IDs listed in the description, tests added or updated, screenshots at mobile and desktop for UI changes, section 11 items affected are ticked, documentation updated.

Scaling roadmap

Task	Frontend change	Documents to update	Gate additions
2. Backend API	Add <code>adapters/http</code> ; generate types from the FastAPI OpenAPI schema; session in <code>HttpOnly</code> cookie; same scenarios replayed against the API	SRS (API requirements), threat model TH-08, ADR-008	Contract tests, API status codes and error envelope checks
3. Database	No component change; add pagination and server-side search; verify optimistic updates against real persistence	Domain model, data model and ERD, NFR-24 evidence	Migration and data integrity checks, <code>large</code> scenario against real data
4. AI platform	AI feature with streaming, cancel, and failure states added to the state matrix; output rendered as plain text	UX spec (AI states), threat model TH-10, new NFRs for latency and cost	LLM Top 10 review, prompt-injection tests, cost and rate-limit checks
Beyond the internship	Roles and permissions, i18n languages, error monitoring, feature flags, Storybook, extracted design-system package	Full SRS revision, runbook, on-call and incident process	Load tests, monitoring alerts, release checklist

Change control. A new or changed requirement gets a new ID or a version note, its tests are added in the same pull request, and the traceability matrix in section 9 is updated so it never drifts from the code.